

# **DSC 204A: Last Lecture**

**Haojian Jin**

## **Agenda**

1. Fill evaluation form.
2. How to prepare for technical interviews today?
3. The research problems I am working on.

# Part 1

## How to prepare for technical interviews today?

- Two types of companies and three types of questions
- What interviewers are looking for?
- Getting interviews?
- Negotiating offers
- Cultural fit
- How to answer "do you have questions?"

## Stories & Caveat

- order fish
- buy single family houses | fidelity zero | ...
-

## Two types of companies

### Big Tech / Product

- **FAANG**, established firms
- Hire for **general ability** – they'll figure out your team later
- Structured, predictable loops
- You're a strong cog in a big machine

### Startups / Scale-ups

- Hire for a **specific gap** they need filled *now*
- Loops vary wildly – sometimes a take-home, sometimes a chat with the founder
- You wear many hats
- Higher risk, higher ownership

Same resume, **two completely different games**. Know which one you're playing.

## Three types of questions

1. **Coding / Algorithms** – can you turn a problem into correct, efficient code under pressure?
2. **System Design** – can you reason about scale, trade-offs, and the unknown?
3. **Behavioral** – can I work with you? Will you grow?

For a **master's grad**, the weighting shifts by role:

- ML / Data roles → expect ML system design + a coding round
- SWE roles → coding is king, design grows with seniority

## What are interviewers actually looking for?

They are **not** only scoring whether you got the answer. They're collecting **signal**:

- **Problem-solving process** — how do you move when you're stuck?
- **Communication** — can you make me follow your thinking?
- **Code/design quality** — is this someone whose work I'd trust?
- **Coachability** — do you take a hint gracefully?

## Getting interviews: the hidden funnel

The application is the part people overweight. The **funnel** is where you win.

- Cold applications: ~**2%** response
- Referrals: ~**10-30%** — *aim here*
- Recruiter reach-outs: warmest of all

Don't "only" apply *harder*.

### What actually moves the needle:

- A **referral** from anyone who has met you
- A **project** recruiters can click and see
- Showing up where engineers are (OSS, meetups, papers)



## Your resume gets ~6 seconds

Make those seconds count:

- Key words for passing the machine filtering.
- **Impact, not duties** – "cut inference latency **40%**", not "worked on inference"
- **Numbers everywhere** – scale, speedups, users, accuracy
- **One page**, reverse-chronological
- Tailor the **top third** to the role you're targeting

## Negotiating offers — yes, you should

The single highest-ROI hour of your job search. **The first number is rarely the best number.**

- The offer is a **signal they want you** — leverage exists
- Negotiating (politely) **does not** rescind offers — that's a myth
- Total comp = **base + bonus + equity + sign-on** — every line is negotiable

"I'm really excited about this team. The offer is a bit below what I was hoping for — is there flexibility on the base or sign-on?"

## Negotiation tactics that work

1. **Never give the first number** – "What's the budgeted range for this level?"
2. **Get it in writing** before you respond
3. **Create leverage** – **competing offers** are the strongest; a stated timeline is good too
4. **Negotiate the whole package** – if base is capped, ask for sign-on or equity
5. **Stay warm and collaborative** – you'll work *with* these people

Silence is a tool. After they make an offer, it's okay to say "let me think it over."

## Which rung of the ladder are you joining?

Comp isn't one number — it's a **band tied to your level**. Negotiate the *level*, not just the salary.

- Google: **L3** → **L4** → **L5** (SWE II → Senior)
- Amazon: **SDE I** → **II** → **III**
- Meta: **E3** → **E4** → **E5**
- A master's often maps to **one rung above** new-grad
- Each rung = a ~**20-40% comp jump**
- Same title means *different things* at different firms

Before you accept, ask: "*What level is this, and what does the next one look like?*"

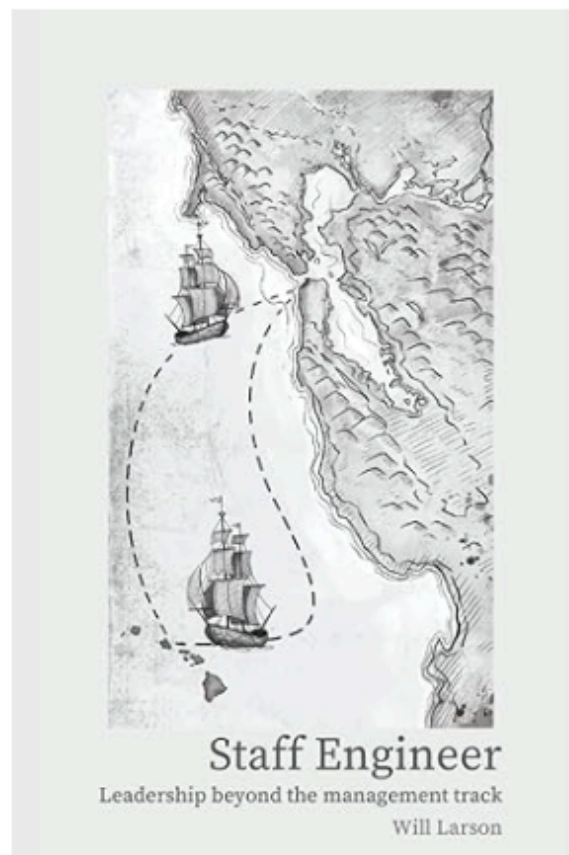
## The scope of different levels

What changes as you climb isn't *harder* code — it's the **size of the ambiguity you own**.

| Level                     | Scope                   | The question you answer                  |
|---------------------------|-------------------------|------------------------------------------|
| <b>Junior</b> (L3 / E3)   | A <b>task</b>           | "How do I build <i>this</i> , well?"     |
| <b>Mid</b> (L4 / E4)      | A <b>feature</b>        | "What's the right way to build this?"    |
| <b>Senior</b> (L5 / E5)   | A <b>project / team</b> | " <i>What</i> should we build, and why?" |
| <b>Staff+</b> (L6+ / E6+) | An <b>org / domain</b>  | "Where is the whole team going?"         |

You get promoted by **already operating** at the next level — then the title catches up.

 Purchased on Aug 14, 2025 [View order](#) | [View Your Books](#) | [Discover more](#)



## Staff Engineer: Leadership beyond the management track Paperback

– January 31, 2021

by [Will Larson](#) (Author), [Tanya Reilly](#) (Foreword)

4.4 ★★★★★ (1,211)

 No better price found

4.0 on Goodreads 3,153 ratings

[See all formats and editions](#)

At most technology companies, you'll reach Senior software engineer, the career level for software engineers, in five to eight years. At the career level, your company's career ladder won't require that you work towards the next promotion; being promoted further is an exception rather than expected. This is also when many engineers are first given an opportunity to move into engineering management. Over the past few years, we've seen a flurry of books unlocking the engineering management career path, like Camille Fournier's *The Manager's Path*, Julie Zhuo's *The Making of a Manager*, Lara Hogan's *Resilient Management*, and even my own *An Elegant Puzzle*. The engineering management career isn't an easy one, but there are maps available to help navigate it. What if you want to advance your career without becoming an engineering manager? The technical leadership path remains relatively undocumented, hard to navigate, and inconsistent across companies. *Staff Engineer* is your guide to building your career towards a Staff engineering role, receiving the title, and succeeding within the role.

 [Report an issue with this product or seller](#)



[Read sample](#)

[▶ Audible sample](#)

Print length

1

371 pages

Language



English

Publication date



January 31, 2021

Dimensions



6 x 0.84 x 9 inches

ISBN-10



1736417916



[See all details](#)

## Cultural fit (it goes both ways)

Companies screen for fit. **So should you** – you'll spend years here.

- Behavioral rounds probe: ownership, conflict, failure, collaboration
- Use the **STAR** frame: Situation → Task → Action → Result
- Have **3-4 stories** ready that flex to different questions

### Reverse-interview them:

- "What does success look like in 6 months?"
- "Why is this role open?"
- "How does the team handle disagreement?"

## Use Amazon as an example

Amazon's entire behavioral loop runs on its **16 Leadership Principles (LPs)** – and a **Bar Raiser** whose only job is to protect the hiring bar.

### The culture (the LPs that show up most)

- **Customer Obsession** – start from the customer, work backwards
- **Ownership** – "not my job" is the wrong answer
- **Dive Deep** – know your numbers, no hand-waving
- **Bias for Action** – speed matters; decisions are reversible
- **Disagree and Commit** – argue hard, then row together

### How to ace it

- Bring **6-8 STAR stories**, each pre-mapped to 2-3 LPs
- Say **"I"**, not "we" – they score *your* actions
- **Quantify** every result (%, \$, latency, users)
- Prepare a real **failure / conflict** story – they *will* ask
- When they **Dive Deep**, go one layer deeper than feels natural

The interviewer literally has an LP rubric open. Name the behavior, then prove it with a metric.



## Anatomy of a STAR story

A structure that forces you to show what **you** did — not what your team did.

- **S — Situation:** set the scene in 1-2 sentences
- **T — Task:** what you specifically owned
- **A — Action:** what **you** did, step by step (*spend ~70% here*)
- **R — Result:** how it ended — **quantified**

**S:** "Our nightly data pipeline was failing before a launch."

**T:** "I was on call and owned getting reliability back."

**A:** "I traced it to a memory leak, added retries, and rewrote the batching."

**R:** "Failures → **0**, pipeline ran **2× faster**, we shipped on time."

Say "**I**," not "**we**." End on a number. Pre-write **6-8 stories** that flex across questions.

## How to answer "do you have questions?"

**Never say "no."** This is the easiest place to score points — and the only one *you* control.

### Bad / wasted

- "No, I think you covered everything."
- Asking what you could Google (revenue, headcount)
- Generic "what's the culture like?"

### Good — shows you'd thrive here

- "What's the **hardest problem** the team is facing right now?"
- "What separates someone who's **good** here from **great**?"
- "What would you want me to **ship in my first 90 days**?"

A sharp question signals you're *already thinking like a teammate*, not a candidate. Leave them remembering it.

## Takeaways

- **Know which game** you're playing – big tech vs. startup, three question types
- Interviewers buy **signal**, not answers – narrate your thinking
- Get interviews **warm**: referrals + a clickable project
- **Always negotiate** – politely, on the whole package
- Fit is **mutual** – interview them back

You are not begging for a job. You are finding a **two-way match**.

## **Part 2**

**The research problems I am working on**

## Five questions I can't stop thinking about

1. How will AI **reshape the job market**?
2. How do we protect **vulnerable populations** in the age of AI?
3. What is **AI safety** when AI surpasses us?
4. How do we win **AI security** – both sides of the arms race?
5. Can AI *help* **privacy** more than it threatens it?

| The thread connecting all five: agents are arriving faster than our institutions can adapt.

## How will AI reshape the job market?

If AI is becoming the economy's core input, we should **measure its price** the way we measure everything else.

### Intelligence Price Index

- An analogy to the **Consumer Price Index (CPI)**
- Track the falling cost of a "unit of intelligence" over time
- A macro signal for how fast work is being automated

### Emerging roles to train for

- **Benchmark engineer** – define what "good" even means
- **Agent operator** – run fleets of agents in production
- **Harness engineer** – build the scaffolding agents act through
- **Simulation engineer** - simulation v.s. prediction

The jobs don't disappear – they **move up the stack**. Who trains people for the new ones?

## AI for vulnerable populations

The people most exposed to generative AI's harms have the **least power** to defend themselves.

- **Children:** generative AI with no guardrails — what does a *parental control* layer look like?
- Beyond a blocklist: **context-aware** protection that understands intent, not just keywords
- The elderly, low-literacy users, non-native speakers — all exposed to new manipulation

| We childproofed the web. We have **not** childproofed the model.

## AI safety when AI surpasses us

Most safety work assumes humans stay in the loop. **What if they can't keep up?**

- Research that *assumes* AI will exceed human intelligence – and plans for it now
- We need ways to **evaluate systems smarter than their evaluators**
- Decentralized, adversarial **arenas** where models stress-test each other



## AI security: both sides of the arms race

A new attacker has entered – and a new thing to defend.

### Defend systems from AI

- AI-powered attackers: automated exploits, scaled social engineering, tireless probing
- Defenses must operate at **machine speed**, not human speed

Every capability we give an agent is also a door we just unlocked.

### Defend the AI itself

- Prompt injection, data poisoning, model exfiltration
- The agent's tools and memory are a **new attack surface**

## AI privacy: threat *and* cure

Agentic workflows break the old privacy model – but AI can also **rebuild** it.

- **The new risk:** agents read your inbox, calendar, files – and act across them. Data flows nobody mapped.
- **The opportunity:** use AI to *strengthen* the privacy ecosystem – auto-redaction, purpose auditing, consent that a machine can actually enforce
- Toward agents that are **privacy-preserving by construction**, not by policy

| The same model that can leak everything can also be the thing that guards it.

## Thank you — let's keep in touch

- Fill out the **course evaluation** 🙏
- These problems are **open** — come work on them with me